

wp-06-concept-directional-substitution

Provenance-Tracked Concept Directional Substitution: Lineage-Sourced Replacement Vectors for Auditable Model Editing

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: May 2026 · **Revision 2:** June 2026

Working Paper Series: AAMT-WP-06

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Revision 2 changes: (C7) §2.2 now requires the substitution direction to be orthogonal to the removed direction ($\langle \mathbf{d}_{\text{sub}}, \mathbf{d} \rangle = 0$), enforced by a Gram–Schmidt step, with a proof that without it the substitution silently re-introduces a component of the ablated concept. No other content changed.

Abstract

Model editing via concept direction removal — projecting out mean-difference probed directions from attention output weight matrices — has emerged as a practical technique for suppressing unwanted behaviors in fine-tuned language models (Arditi et al., 2024). Existing work focuses on the removal half of the operation; the replacement half is left to the model’s residual behavior after removal. We present **Provenance-Tracked Concept Directional Substitution (PT-CDS)**, which extends mean-difference probing with a second operation: injecting a lineage-sourced replacement direction harvested from the operator’s own training lineage graph. Every edit is recorded in a cryptographically hashed `concept_edit_ledger.json` file that stores the concept name, targeted layer indices, direction SHA-256, edit strength, and the lineage thread IDs that sourced the replacement. The ledger enables full reproducibility (any edit can be replayed from the hashes), revertibility (the original weight matrix can be reconstructed), and attribution (each edited behavior traces to specific training lineage content). PT-CDS includes a perplexity-gated safety guard: if the post-edit perplexity on a held-out neutral set exceeds `max_ppl_ratio × baseline_ppl` (default 2.0×), the edit is reverted automatically. PT-CDS is specified as Tier-1 feature T1.1 of MaiaM Alchemist.

Keywords: model editing, representation engineering, ablation, concept direction, lineage tracking, auditable AI, mean-difference probing, LoRA, weight manipulation

1. Introduction

Post-training model editing — modifying a trained model’s behavior without full retraining — has become an important practical tool. Two paradigms dominate: **weight editing** (ROME, MEMIT: directly modify weights to change factual associations) and **representation engineering** (Zou et al., 2023: add or subtract concept directions from the residual stream at inference time). A third approach, popularized in 2024 as “ablation” (Arditi et al., 2024), removes concept directions from the static weight matrices of attention output projections, making the behavior change permanent without inference overhead.

All three paradigms share a limitation: they specify what to remove but not what to replace it with. After removal, the model’s behavior in the edited region is determined by whatever residual structure remains — often unpredictable and potentially incoherent. For operators who have a clear positive replacement in mind (e.g., “replace deception patterns with truthfulness patterns from my own lineage”), there is no standard mechanism.

PT-CDS addresses this with a two-phase edit pipeline:

1. **Remove:** project out the target concept direction from the weight matrices (standard mean-difference ablation)
2. **Substitute:** inject a replacement direction sourced from the operator’s lineage graph as a low-rank delta

The combination is what makes PT-CDS novel — not the removal technique (which follows Arditi et al., 2024) but the lineage-sourced substitution and the cryptographic audit ledger.

2. Technical Description

2.1 Mean-Difference Direction Probing

For each target concept c and each probe layer l :

1. Collect N exemplars rated HeartScale-low for concept c (e.g., outputs with high deception signal).
2. Collect N exemplars rated HeartScale-high for concept c (e.g., outputs with low deception, high truthfulness).
3. Forward both sets through the model; collect activations at layer l ’s `o_proj` input.
4. Compute:

```
low_mean = mean_activation(low_exemplars, layer=l)
high_mean = mean_activation(high_exemplars, layer=l)
direction = normalize(low_mean - high_mean) # unit vector in activation space
```

5. For each weight matrix W in the targeted layer's attention output projection:

```
W_edited = W - ablation_strength * outer(direction, direction @ W)
# Equivalent to:  $W \leftarrow W - \text{direction} \cdot (\text{direction}^T \cdot W)$ 
# This projects out the direction from W's column space
```

This operation makes the weight matrix orthogonal to direction in the sense that activations along direction no longer contribute to the output.

2.2 Lineage-Sourced Substitution

The replacement direction is harvested from the operator's lineage graph. For a target lineage thread (e.g., `substitute_from_lineage: ["amitabha_lineage", "harmonic_reasoning"]`):

```
lineage_examples = lineage.query(
    kind="chat.assistant_reply",
    tags=["amitabha_lineage"],
    since=run_start
)
lineage_activations = [get_activation(ex.text, layer=l) for ex in lineage_examples]
raw_sub = normalize(mean(lineage_activations))

# Orthogonalize the substitution against the removed direction d (Gram-Schmidt).
# Without this step the substitution silently re-introduces a component of the
# concept that was just ablated (see correctness condition below).
substitute_direction = raw_sub - (raw_sub @ direction) * direction
substitute_direction = normalize(substitute_direction)
```

The substitution is applied as an additive low-rank delta:

```
delta = substitution_scale * outer(substitute_direction, substitute_direction)
W_final = W_edited + delta # Add substitute direction to the edited matrix
```

This is a rank-1 delta: it adds a single direction to the weight matrix's column space. Merged onto the LoRA as a separate "lineage-delta" adapter, it is distinguishable from the base LoRA and can be removed independently.

Correctness condition (orthogonality). The substitution direction must satisfy $\langle d_{\text{sub}}, d \rangle = 0$. To see why, write the removed direction as the unit vector d and note that after removal $d^T W_{\text{edited}} = d^T W - (d^T d)(d^T W) = 0$. Adding the rank-1 substitution gives

```
d^T W_final = d^T W_edited + substitution_scale * (d^T d_sub) * d_sub^T
              = substitution_scale * (d^T d_sub) * d_sub^T.
```

If $\langle d_{\text{sub}}, d \rangle \neq 0$, the edited matrix regains sensitivity correlated with the removed direction in proportion to $\text{substitution_scale} \cdot \langle d_{\text{sub}}, d \rangle$, partially undoing the ablation. Enforcing $\langle d_{\text{sub}}, d \rangle = 0$ (the Gram–Schmidt step above) drives this term to zero, so the remove and substitute operations are independent. Empirically, with a random unit d_{sub} the re-introduced magnitude is $\text{substitution_scale} \cdot |\langle d_{\text{sub}}, d \rangle|$, which is non-negligible (≈ 0.3 at $\text{substitution_scale} = 0.5$ in dimension 8); orthogonalization removes it exactly.

2.3 The Concept Edit Ledger

Every edit is recorded deterministically:

```
{
  "concept": "deception",
  "layer": 16,
  "direction_sha256": "a3f2c1...",
  "ablation_strength": 1.0,
  "substitution_scale": 0.5,
  "substitute_direction_sha256": "b7e4d2...",
  "substitute_orthogonalized": true,
  "val_ppl_before": 12.4,
  "val_ppl_after": 14.1,
  "ppl_ratio": 1.14,
  "lineage_thread_ids": ["chat-2026-04-12-abc", "chat-2026-05-01-xyz"],
  "timestamp": "2026-05-28T14:22:04Z",
  "reverted": false
}
```

The direction hashes are SHA-256 of the serialized direction vector (float32, little-endian). Any party with access to the model weights can recompute these hashes to verify that the recorded direction matches what is actually embedded in the weights. The `substitute_orthogonalized` flag records that the §2.2 correctness condition was enforced.

2.4 Perplexity Safety Guard

After each edit, the modified model is evaluated on a held-out neutral set (examples that do not contain the target concept):

```
ppl_after = evaluate_perplexity(edited_model, neutral_holdout_set)
ppl_ratio = ppl_after / ppl_baseline # ppl_baseline computed before the edit

if ppl_ratio > config.max_ppl_ratio: # default 2.0
    revert_edit(W_original)
    ledger.mark_reverted(edit_id)
    raise AbliterationGuardError(
        f"Edit reverted: perplexity ratio {ppl_ratio:.2f} exceeds "
```

```
f"max_ppl_ratio {config.max_ppl_ratio:.2f}"
)
```

The `ppl_baseline` is computed on the same neutral set with the same model **before** the edit begins, ensuring the comparison is properly paired.

3. Configuration Schema

```
class AbliterationConfig(BaseModel):
    enabled: bool = False
    target_concepts: list[str] = []
    substitute_from_lineage: list[str] = []
    probe_layer_range: tuple[int, int] = (8, 24)
    probe_token_pool: Literal["last", "mean", "max"] = "mean"
    probe_n_examples_per_concept: int = 64
    abliteration_strength: float = 1.0
    substitution_scale: float = 0.5
    orthogonalize_substitution: bool = True # enforce (d_sub, d) = 0 (§2.2)
    ppl_baseline: float | None = None # computed if None
    max_ppl_ratio: float = 2.0
    ledger_path_relative_to_run: str = "concept_edit_ledger.json"
```

4. Stage Position in the Training Pipeline

PT-CDS runs as a post-training stage: after `train`, before `export`. The edited model is then exported to ONNX and GGUF with the substitution delta merged.

```
load_corpus → preprocess → load_model → train → abliterate (PT-CDS) → watermark → export
```

The post-training position is deliberate: the ablation operates on the fully fine-tuned model, not on the base model. Fine-tuning may have introduced or amplified the target concept direction; PT-CDS corrects it after the fact without requiring full retraining.

5. Relationship to Prior Work

ROME / MEMIT (Meng et al., 2022; Meng et al., 2023): Edit factual associations by directly modifying FFN weights. PT-CDS edits behavioral directions in attention projections, not factual associations.

RepE (Zou et al., 2023): Inference-time representation engineering: add/subtract concept vectors from the residual stream. PT-CDS modifies weights statically — zero inference overhead.

Abliteration (Arditi et al., 2024): Mean-difference probing + direction removal from `o_proj` weights. PT-CDS extends this with lineage-sourced substitution (orthogonalized to the removed direction) and a cryptographic audit ledger.

Model editing surveys (Yao et al., 2023): No existing survey includes a lineage-sourced replacement mechanism or a cryptographic audit trail.

Key distinction: PT-CDS is the first model editing technique that (1) replaces removed directions with operator-specific content from a traceable lineage graph, orthogonally so the removal is preserved, and (2) generates a cryptographically verifiable audit record of every weight modification.

6. Limitations

- Rank-1 substitution capacity:** A single rank-1 delta may not fully capture the desired replacement behavior if that behavior spans multiple directions. The `substitution_scale` parameter can be tuned, and multiple substitution passes (one per direction, each orthogonalized against the removed direction and against prior substitutions) can be chained.
 - Neutral set quality:** The perplexity guard’s effectiveness depends on the quality of the neutral holdout set. If the neutral set is small or atypical, the guard may not catch problematic edits.
 - Layer range sensitivity:** The probing layer range (default layers 8–24 of a 26-layer model) captures mid-stack representations. Very shallow or very deep edits may have different behavioral effects not captured by this range.
-

7. Conclusion

PT-CDS provides an end-to-end auditable model editing pipeline: remove concept directions via mean-difference probing, inject lineage-sourced replacement directions orthogonal to what was removed, and record every operation in a cryptographically hashed ledger. The orthogonality condition guarantees the remove and substitute operations are independent; the perplexity guard prevents edits that degrade general language modeling quality; the ledger enables compliance review, edit replay, and behavioral attribution. This combination makes model editing a first-class, auditable operation within the closed-loop improvement system of MaiaM Alchemist.

Acknowledgments

The author thanks the AsAManThinks Research community.

Funding

Self-funded through AsAManThinks Research.

Data Availability

The PT-CDS stage implementation is part of the MaiaM Alchemist training pipeline (internal). The concept_edit_ledger schema is fully described herein and is compatible with any JSON-capable audit system.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

- Arditi, A., Obeso, O., Paleka, D., Panickssery, N., Gat, I., & Hernandez-Orallo, J. (2024). Refusal in language models is mediated by a single direction. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2406.11717>
- Meng, K., Bau, D., Andonian, A. J., & Belinkov, Y. (2022). Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems 35* (pp. 17359–17372). <https://doi.org/10.48550/arXiv.2202.05262>
- Meng, K., Sharma, A. S., Andonian, A., Belinkov, Y., & Bau, D. (2023). Mass-editing memory in a transformer. In *Proceedings of ICLR 2023*. <https://doi.org/10.48550/arXiv.2210.07229>
- Yao, Y., Wang, P., Tian, B., Cheng, S., Li, Z., Deng, S., Chen, H., & Zhang, N. (2023). Editing large language models: Problems, methods, and opportunities. In *Proceedings of EMNLP 2023* (pp. 10222–10240). <https://doi.org/10.18653/v1/2023.emnlp-main.632>
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Schwartz, S., Bhatt, U., Goldwasser, D., Kambhampati, S., ... Hendrycks, D. (2023). Representation engineering: A top-down approach to AI transparency. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2310.01405>
- Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>